Funkcje na napisach w C

Ściąga do SOP w formie wyjaśnień: co robi każda funkcja, co zwraca, przykład, zastosowanie i pułapki.

Zasada ogólna: funkcje `str*` działają do pierwszego `\0` i zakładają, że dane to napis. Funkcje `mem*` działają na dokładnie `n` bajtach i ignorują `\0`, więc nadają się do danych binarnych. Poza `strdup`/`strndup` (brak pamięci) i `strtol` (`ERANGE`) żadna z tych funkcji nie zgłasza błędów, więc ich wyniku nie sprawdzasz pod kątem `ERR`.

Długość

strlen <string.h>

Prototyp: `size_t strlen(const char *s);`

Co robi: Liczy znaki napisu od początku aż do kończącego `\0` (samego `\0` nie wlicza).

Zwraca: Długość jako `size_t`. Nie zgłasza błędów.

```
char *copy = malloc(strlen(s) + 1);   /* +1 na '\0' */
```

Zastosowanie:

- Obliczenie, ile pamięci zaalokować na kopię lub sklejony napis — tak liczy rozmiar `join_paths`.
- Długość klucza do `strncmp` (`strlen("title:")`), żeby nie liczyć ręcznie.

Pułapki:

- Bez `+ 1` przy `malloc` zabraknie miejsca na `\0`.
- `sizeof` to nie `strlen`: dla parametru funkcji daje rozmiar wskaźnika (8 bajtów).
- W warunku pętli liczy długość od nowa w każdym obrocie — policz raz przed pętlą.

Kopiowanie

strcpy <string.h>

Prototyp: `char *strcpy(char *dst, const char *src);`

Co robi: Kopiuje cały napis `src` razem z `\0` do bufora `dst`.

Zwraca: Wskaźnik `dst`. Nie zgłasza błędów.

```
char *res = malloc(strlen(p) + 1);
if (!res) ERR("malloc");
strcpy(res, p);
```

Zastosowanie:

- Kopiowanie do bufora, o którym wiesz, że jest wystarczająco duży, np. świeżo zaalokowanego na podstawie `strlen`.

Pułapki:

- Nie sprawdza rozmiaru `dst` — za mały bufor to przepełnienie pamięci.
- `dst` nie może być literałem ani niezainicjalizowanym wskaźnikiem.

strncpy <string.h>

Prototyp: `char *strncpy(char *dst, const char *src, size_t n);`

Co robi: Kopiuje najwyżej `n` znaków. Jeśli `src` jest krótszy, dopełnia resztę zerami.

Zwraca: Wskaźnik `dst`. Nie zgłasza błędów.

```
char title[65];
strncpy(title, src, 64);
title[64] = '\0';      /* strncpy mogło go nie dopisać */
```

Zastosowanie:

- Wpisywanie napisu do pola stałej długości, np. 64-bajtowego tytułu w rekordzie binarnym — dokładnie taki format ma baza z zadania.

Pułapki:

- Gdy `src` ma `n` lub więcej znaków, **nie dopisuje** `\0` — terminator ustawiasz sam.
- Nie traktuj go jak „bezpiecznego strcpy” — do kopii z limitem lepsze są `strndup` albo `snprintf`.

strdup <string.h>

Prototyp: `char *strdup(const char *s);`

Co robi: Alokuje nową pamięć (przez `malloc`) i kopiuje do niej napis. Dostajesz niezależną kopię.

Zwraca: Wskaźnik na kopię albo `NULL` przy braku pamięci.

```
char *name;
if ((name = strdup(dp->d_name)) == NULL) ERR("strdup");
/* ... */
free(name);
```

Zastosowanie:

- Zachowanie `d_name` z `readdir` na później — bufor wpisu zostanie nadpisany przy kolejnym `readdir`.
- Kopiowanie linii wczytanych przez `fgets` do tablicy, jak w `GetText`.

Pułapki:

- Każdą kopię trzeba zwolnić przez `free`, inaczej wyciek pamięci.
- Kopiuje tylko do pierwszego `\0` — nie do danych binarnych.

strndup <string.h>

Prototyp: `char *strndup(const char *s, size_t n);`

Co robi: Jak `strdup`, ale kopiuje najwyżej `n` bajtów i zawsze dopisuje `\0`.

Zwraca: Wskaźnik na kopię albo `NULL` przy braku pamięci.

```
char *title;
if ((title = strndup(line + strlen("title:"), 64)) == NULL)
    ERR("strndup");
```

Zastosowanie:

- Kopia z przycięciem — tytuł i gatunek obcięte do 64 bajtów w zadaniu z biblioteką.
- Wycięcie fragmentu napisu, gdy długość znasz np. z `strcspn`.

Pułapki:

- Tak jak `strdup`: `free` i sprawdzanie `NULL`.
- `n` liczy bajty — znak UTF-8 (np. grecki) może zostać przecięty w połowie.

memcpy <string.h>

Prototyp: `void *memcpy(void *dst, const void *src, size_t n);`

Co robi: Kopiuje dokładnie `n` bajtów, bez względu na ich treść — `\0` nie ma tu żadnego znaczenia.

Zwraca: Wskaźnik `dst`. Nie zgłasza błędów.

```
memcpy(buf, title, len);
buf[len] = '\0';       /* przy tekście terminator sam */
```

Zastosowanie:

- Kopiowanie danych binarnych i struktur, bloków wczytanych przez `fread` lub `readv`.

Pułapki:

- Nie dopisuje `\0`.
- Obszary nie mogą się nakładać — wtedy używasz `memmove`.

memmove <string.h>

Prototyp: `void *memmove(void *dst, const void *src, size_t n);`

Co robi: Jak `memcpy`, ale działa poprawnie także wtedy, gdy `src` i `dst` się nakładają.

Zwraca: Wskaźnik `dst`. Nie zgłasza błędów.

```
/* usuń 2 pierwsze znaki napisu */
memmove(s, s + 2, strlen(s + 2) + 1);
```

Zastosowanie:

- Przesuwanie danych w obrębie jednego bufora, np. usunięcie początku napisu.

Pułapki:

- Przy przesuwaniu napisu przenieś też `\0` (stąd `strlen + 1`).

Doklejanie

strcat <string.h>

Prototyp: `char *strcat(char *dst, const char *src);`

Co robi: Dopisuje `src` na koniec napisu w `dst` (od miejsca, gdzie jest jego `\0`).

Zwraca: Wskaźnik `dst`. Nie zgłasza błędów.

```
char path[PATH_MAX];
strcpy(path, dir);
strcat(path, "/");
strcat(path, name);
```

Zastosowanie:

- Sklejanie ścieżek: katalog + `/` + nazwa pliku — ostatni krok w `join_paths`.

Pułapki:

- `strcat(".", name)` pisze do literału — segfault. To był błąd w pierwszej wersji `SearchLibrary`.
- `dst` musi mieć miejsce na wynik i być już poprawnym napisem.
- Modyfikuje `dst` w miejscu — doklejenie do `cwd` zepsuło jego zawartość w `MakeIndex`.

strncat <string.h>

Prototyp: `char *strncat(char *dst, const char *src, size_t n);`

Co robi: Dopisuje najwyżej `n` znaków z `src` i zawsze kończy wynik `\0`.

Zwraca: Wskaźnik `dst`. Nie zgłasza błędów.

```
strncat(path, name, sizeof(path) - strlen(path) - 1);
```

Zastosowanie:

- Doklejanie do bufora stałego rozmiaru, gdy dane mogą być za długie.

Pułapki:

- `n` to limit **dopisywanych** znaków, a nie rozmiar bufora.

Porównywanie

strcmp <string.h>

Prototyp: `int strcmp(const char *a, const char *b);`

Co robi: Porównuje dwa napisy znak po znaku, aż do różnicy albo końca.

Zwraca: `0` gdy równe, wartość ujemną lub dodatnią przy różnicy. Nie zgłasza błędów.

```
if (strcmp(dp->d_name, ".") == 0 ||
    strcmp(dp->d_name, "..") == 0) continue;
```

Zastosowanie:

- Pomijanie `.` i `..` przy `readdir` — bez tego rekurencja się zapętla.
- Szukanie pliku o konkretnej nazwie.

Pułapki:

- Równość to `0`, czyli fałsz w warunku — zawsze pisz `== 0`.
- `a == b` porównuje adresy, nie treść.
- Napis z `\n` po `fgets` nie jest równy temu samemu bez `\n`.

strncmp <string.h>

Prototyp: `int strncmp(const char *a, const char *b, size_t n);`

Co robi: Porównuje najwyżej `n` pierwszych znaków — w praktyce sprawdza prefiks.

Zwraca: Jak `strcmp`: `0` gdy pierwsze `n` znaków jest równe.

```
if (strncmp(line, "title:", strlen("title:")) == 0)
    value = line + strlen("title:");
```

Zastosowanie:

- Sprawdzanie, czy linia zaczyna się od klucza (`title:`, `author:`) — tak odrzuciliśmy `latin_title:` i `title_gr:`.

Pułapki:

- Za małe `n` porównuje za mało: z `5` przejdzie `title_gr:`. Używaj `strlen(klucz)`.
- Dołącz `:` do klucza, inaczej `title` dopasuje też `titles`.

strcasecmp <strings.h>

Prototyp: `int strcasecmp(const char *a, const char *b);`

Co robi: Jak `strcmp`, ale bez rozróżniania wielkości liter.

Zwraca: Jak `strcmp`.

```
if (strcasecmp(ext, ".txt") == 0) { /* .txt, .TXT, .Txt */ }
```

Zastosowanie:

- Porównanie rozszerzeń plików lub słów wpisanych przez użytkownika.

Pułapki:

- Nagłówek to `<strings.h>`, z „s” na końcu.
- Działa tylko dla liter ASCII.

memcmp <string.h>

Prototyp: `int memcmp(const void *a, const void *b, size_t n);`

Co robi: Porównuje dokładnie `n` bajtów, nie zatrzymując się na `\0`.

Zwraca: Jak `strcmp`.

```
if (memcmp(buf1, buf2, n) == 0) { /* identyczne */ }
```

Zastosowanie:

- Porównanie danych binarnych, np. czy dwa wczytane bloki są takie same.

Pułapki:

- Struktury mogą różnić się bajtami paddingu, nawet przy równych polach.

Wyszukiwanie

strchr <string.h>

Prototyp: `char *strchr(const char *s, int c);`

Co robi: Szuka pierwszego wystąpienia znaku `c` w napisie.

Zwraca: Wskaźnik na znaleziony znak albo `NULL`.

```
char *sep = strchr(line, ':');
if (sep != NULL) value = sep + 1;
```

Zastosowanie:

- Znalezienie separatora, np. `:` w linii `klucz:wartość`.

Pułapki:

- Przed użyciem wyniku sprawdź `NULL`.
- Podajesz znak `':'`, a nie napis `":"`.

strrchr <string.h>

Prototyp: `char *strrchr(const char *s, int c);`

Co robi: Szuka **ostatniego** wystąpienia znaku `c`.

Zwraca: Wskaźnik na znaleziony znak albo `NULL`.

```
char *slash = strrchr(path, '/');
char *name = slash ? slash + 1 : path;
```

Zastosowanie:

- Wyciągnięcie nazwy pliku z pełnej ścieżki albo rozszerzenia (ostatnia kropka).

Pułapki:

- Wynik wskazuje na `/` — nazwa zaczyna się od `wynik + 1`.
- Ścieżka bez `/` daje `NULL`.

strstr <string.h>

Prototyp: `char *strstr(const char *s, const char *sub);`

Co robi: Szuka podnapisu `sub` w dowolnym miejscu napisu `s`.

Zwraca: Wskaźnik na **początek** dopasowania albo `NULL`.

```
char *p = strstr(text, "author");
if (p != NULL) after = p + strlen("author");
```

Zastosowanie:

- Sprawdzenie, czy tekst zawiera dane słowo gdziekolwiek.

Pułapki:

- Najpierw **gdzie** szukać, potem **czego** — zamiana była błędem w `PrintInfo`.
- Znajduje fragment w środku: `title:` w `latin_title:`. Do kluczy na początku linii — `strncmp`.

strpbrk <string.h>

Prototyp: `char *strpbrk(const char *s, const char *set);`

Co robi: Szuka pierwszego znaku, który należy do zbioru `set`.

Zwraca: Wskaźnik na ten znak albo `NULL`.

```
char *p = strpbrk(line, " \t");
if (p != NULL) *p = '\0';   /* utnij na 1. białym znaku */
```

Zastosowanie:

- Znalezienie pierwszego z kilku możliwych separatorów.

Pułapki:

- `set` to zbiór pojedynczych znaków, nie napis.

strcspn <string.h>

Prototyp: `size_t strcspn(const char *s, const char *set);`

Co robi: Liczy, ile znaków na początku `s` **nie należy** do zbioru `set` — czyli indeks pierwszego znaku ze zbioru.

Zwraca: Indeks (`size_t`); gdy nic nie znajdzie — długość napisu.

```
line[strcspn(line, "\n")] = '\0';   /* usuń '\n' po fgets */
```

Zastosowanie:

- Usuwanie `\n` po `fgets` — działa też, gdy `\n` nie ma.
- Długość klucza do separatora, np. `strcspn(line, ":")`.

Pułapki:

- `set` to zbiór znaków, a nie słowo — `strcspn(s, "author")` szuka dowolnej z tych liter.
- Zwraca indeks, nie wskaźnik.

strspn <string.h>

Prototyp: `size_t strspn(const char *s, const char *set);`

Co robi: Odwrotność `strcspn`: liczy znaki na początku, które **należą** do zbioru.

Zwraca: Liczbę takich znaków (`size_t`).

```
value += strspn(value, " \t");   /* pomiń spacje i taby */
```

Zastosowanie:

- Pomijanie spacji lub separatorów na początku wartości, np. w `author: Plutarch`.

Pułapki:

- Łatwo pomylić z `strcspn` — ta liczy znaki **ze** zbioru.

Dzielenie na części

strtok <string.h>

Prototyp: `char *strtok(char *s, const char *delim);`

Co robi: Dzieli napis na kawałki (tokeny) po separatorach. Pierwsze wywołanie z napisem, kolejne z `NULL`.

Zwraca: Wskaźnik na kolejny token albo `NULL`, gdy tokenów brak.

```
for (char *w = strtok(line, " "); w != NULL; w = strtok(NULL, " "))
    puts(w);
```

Zastosowanie:

- Rozbijanie linii na słowa lub pola rozdzielone spacjami czy przecinkami.

Pułapki:

- Modyfikuje napis (wstawia `\0` w miejsce separatorów) — nie działa na literałach.
- Ma jeden ukryty stan — nie da się zagnieździć dwóch podziałów ani używać w wątkach.

strtok_r <string.h>

Prototyp: `char *strtok_r(char *s, const char *delim, char **save);`

Co robi: To samo co `strtok`, ale stan trzyma w twojej zmiennej `save`.

Zwraca: Jak `strtok`.

```
char *save;
for (char *w = strtok_r(line, ",", &save); w != NULL;
     w = strtok_r(NULL, ",", &save))
    puts(w);
```

Zastosowanie:

- Dzielenie w programach wielowątkowych (L3) i podziały zagnieżdżone.

Pułapki:

- Każdy niezależny podział potrzebuje osobnej zmiennej `save`.

Wypełnianie pamięci

memset <string.h>

Prototyp: `void *memset(void *s, int c, size_t n);`

Co robi: Ustawia `n` kolejnych bajtów na wartość `c`.

Zwraca: Wskaźnik `s`. Nie zgłasza błędów.

```
struct book b;
memset(&b, 0, sizeof(b));
```

Zastosowanie:

- Zerowanie bufora lub struktury przed użyciem.

Pułapki:

- `n` to bajty, nie elementy tablicy.
- Ustawia bajty, więc `memset(tab, 1, ...)` na tablicy `int` nie da jedynek.

Tworzenie napisu i konwersje

snprintf <stdio.h>

Prototyp: `int snprintf(char *buf, size_t size, const char *fmt, ...);`

Co robi: Działa jak `printf`, ale wynik trafia do bufora — najwyżej `size - 1` znaków i zawsze `\0`.

Zwraca: Długość, jaką miałby pełny wynik; ujemną przy błędzie.

```
char p[PATH_MAX];
snprintf(p, sizeof(p), "index/by-genre/%s", genre);
```

Zastosowanie:

- Budowanie ścieżek i komunikatów — wygodniejsze niż łańcuch `strcpy` + `strcat`.

Pułapki:

- Wynik `>= size` oznacza, że napis został obcięty.
- Danych od użytkownika nigdy nie podajesz jako `fmt`.

sprintf <stdio.h>

Prototyp: `int sprintf(char *buf, const char *fmt, ...);`

Co robi: To samo co `snprintf`, ale bez limitu rozmiaru.

Zwraca: Liczbę zapisanych znaków.

```
sprintf(buf, "%d", x);   /* tylko gdy bufor na pewno starczy */
```

Zastosowanie:

- W praktyce lepiej zawsze używać `snprintf`.

Pułapki:

- Za długi wynik przepełnia bufor.

strtol <stdlib.h>

Prototyp: `long strtol(const char *s, char **end, int base);`

Co robi: Zamienia napis na liczbę w podanej bazie. `end` wskazuje miejsce, gdzie parsowanie się skończyło.

Zwraca: Liczbę `long`; przy przekroczeniu zakresu ustawia `errno = ERANGE`.

```
mode_t perms = strtol(optarg, NULL, 8);
long size = strtol(optarg, NULL, 10);
```

Zastosowanie:

- Konwersja argumentów z `getopt`: uprawnienia z bazą 8, rozmiar i procent z bazą 10.

Pułapki:

- Uprawnienia zawsze z bazą 8 — `644` dziesiętnie to inne bity.
- `end == s` oznacza, że w napisie nie było liczby.

atoi <stdlib.h>

Prototyp: `int atoi(const char *s);`

Co robi: Zamienia napis na `int` w bazie 10.

Zwraca: Liczbę; dla niepoprawnego napisu zwykle `0`.

```
int n = atoi(argv[1]);
```

Zastosowanie:

- Szybka konwersja danych, o których wiesz, że są poprawne.

Pułapki:

- Nie wykrywa błędów: `"abc"` daje `0`, tak samo jak `"0"`.
- Tylko baza 10 — nie do uprawnień.

Typowe sytuacje

Gotowe połączenia funkcji, które najczęściej pojawiają się w zadaniach z labów.

1. Wczytanie pliku linia po linii bez znaku nowej linii

`fgets` zostawia `\n` w buforze, więc porównania i wypisywanie dają złe wyniki. `strcspn` usuwa go bezpiecznie, także gdy ostatnia linia go nie ma.

```
char line[256];
while (fgets(line, sizeof(line), f) != NULL) {
    line[strcspn(line, "\n")] = '\0';
    /* praca na line */
}
if (ferror(f)) ERR("fgets");
```

2. Rozpoznanie linii klucz:wartość

Klucz sprawdzasz na początku linii razem z separatorem, a wartość to wskaźnik zaraz za nim. Tak odrzuca się `latin_title:` i `title_gr:`.

```
const char *key = "title:";
if (strncmp(line, key, strlen(key)) == 0) {
    char *value = line + strlen(key);
    value += strspn(value, " \t");   /* pomiń spacje po ':' */
    printf("Tytuł: %s\n", value);
}
```

3. Podział linii na klucz i wartość przy dowolnym kluczu

Gdy kluczy jest wiele albo są nieznane, szukasz separatora i rozcinasz linię w miejscu, wstawiając `\0`.

```
char *sep = strchr(line, ':');
if (sep != NULL) {
    *sep = '\0';           /* line to teraz sam klucz */
    char *value = sep + 1;
}
```

4. Pobranie wartości pola z przycięciem do N bajtów

Gdy wartość ma posłużyć np. za nazwę pliku z limitem długości, `strndup` od razu przycina i dopisuje `\0`. Pamiętaj o `free`.

```
char *title;
if ((title = strndup(line + strlen("title:"), 64)) == NULL)
    ERR("strndup");
/* ... */
free(title);
```

5. Sklejenie ścieżki katalog/nazwa

Bufor stałego rozmiaru i `snprintf` są najprostsze. Gdy długość nie jest znana, alokujesz `strlen(a) + strlen(b) + 2` i sklejasz jak w `join_paths`.

```
char path[PATH_MAX];
if (snprintf(path, sizeof(path), "%s/%s", dir, name) >= (int)sizeof(path))
    usage(argv[0]);   /* ścieżka nie zmieściła się w buforze */
```

6. Nazwa pliku i rozszerzenie z pełnej ścieżki

Ostatni `/` oddziela katalog od nazwy, ostatnia `.` nazwę od rozszerzenia. Oba wyszukiwania mogą zwrócić `NULL`.

```
char *slash = strrchr(path, '/');
char *name = slash ? slash + 1 : path;
char *dot = strrchr(name, '.');
if (dot != NULL && strcmp(dot, ".txt") == 0) { /* plik .txt */ }
```

7. Pomijanie . i .. przy przeglądaniu katalogu

Każdy katalog zawiera te dwa wpisy. Bez ich pominięcia rekurencja wchodzi w ten sam katalog bez końca.

```
if (strcmp(dp->d_name, ".") == 0 || strcmp(dp->d_name, "..") == 0)
    continue;
```

8. Zachowanie nazwy z readdir na później

`d_name` leży w buforze wewnątrz `DIR*` i zostanie nadpisane przy kolejnym `readdir` albo zwolnione przy `closedir`.

```
char *names[MAX];
int n = 0;
/* w pętli po readdir: */
if ((names[n++] = strdup(dp->d_name)) == NULL) ERR("strdup");
/* po użyciu: free(names[i]) dla każdego i */
```

9. Napis z pola stałej długości w danych binarnych

Pole tytułu w bazie ma 64 bajty i przy pełnej długości nie ma `\0`. Bufor o 1 większy z wyzerowanym ostatnim bajtem zawsze daje poprawny napis.

```
struct book { unsigned int size; char title[65]; } b;
/* wczytaj 4 bajty do b.size i 64 bajty do b.title */
b.title[64] = '\0';
```

10. Liczba z argumentu programu z kontrolą poprawności

`strtol` pozwala wykryć brak liczby i śmieci po niej. Uprawnienia czytasz z bazą 8, rozmiary z bazą 10.

```
char *end;
long size = strtol(optarg, &end, 10);
if (end == optarg || *end != '\0' || size < 0)
    usage(argv[0]);           /* to nie jest poprawna liczba */
```

11. Rozbicie linii na słowa

`strtok` modyfikuje napis, więc działa na buforze (np. z `fgets`), a nie na literale. Kilka separatorów z rzędu liczy jak jeden.

```
char *w = strtok(line, " \t");
while (w != NULL) {
    printf("[%s]\n", w);
    w = strtok(NULL, " \t");
}
```

12. Usunięcie spacji z końca napisu

Białe znaki na końcu psują porównania i nazwy plików. Idziesz od końca i wstawiasz `\0` za ostatnim znakiem niebędącym spacją. Indeks typu `size_t` zmniejszasz ostrożnie, bo nie może zejść poniżej zera.

```
size_t len = strlen(s);
while (len > 0 && (s[len - 1] == ' ' || s[len - 1] == '\t'))
    s[--len] = '\0';
```